

Stage 1 — Building the Data Layer

A search-engine data layer for Project Gutenberg: datalake, datamarts and control layer, implemented and benchmarked in Python, Java and C++

Course	Big Data — Grado en Ciencia e Ingeniería de Datos (ULPGC)
Academic year	2026/2027
Project title	Stage 1 — Building the Data Layer
Group name	TheScratchers
GitHub repository	https://github.com/TheScratchers/stage_1
Date	October 2026

Group member (full name)	Student ID	Implementation
Aimar Daniel Alejandro Santana	_____	Python — branch <code>ft/python-setup</code>
Amado Artiles Rodríguez	_____	Java — branch <code>ft/java-setup</code>
Pablo Martínez Suárez	_____	C++ — branch <code>ft/c++-setup</code>

Source code, benchmark scripts and raw results (JSON) are in the repository above: `python/`, `java/`, `c++/` and `shared/`.

1. Introduction and objectives

Project Gutenberg offers tens of thousands of public-domain books. A search engine over them needs, before it can answer a single query, a **data layer**: a place where the raw books are stored, derived structures that answer questions quickly, and a process that keeps both consistent. Stage 1 of the project builds that layer. This report describes it, justifies the design choices and measures them.

Scope of the stage. We implement (1) ingestion of books from Project Gutenberg, (2) a *datalake* that keeps every book as a header file and a body file, (3) two *datamarts* derived from it, a metadata store and an inverted index, and (4) a minimal *control layer* that coordinates downloading and indexing without duplicating or losing work. Ranking, a query interface and parallel or distributed ingestion belong to later stages.

Objectives.

1. Implement the data layer with the same architecture in three languages: Python, Java and C++.
2. Compare three datalake layouts (time-, book- and batch-based) and three inverted-index structures (monolithic JSON, hierarchical files and SQLite) at 100, 1,000 and 10,000 books.
3. Make the three implementations comparable through a shared benchmark contract (dataset, tokenizer, queries, scales and replication rule).
4. Stress-test a SQLite metadata datamart up to 100,000 books (a team extension beyond the specification).
5. Select, with evidence, the structures for the final implementation, and state honestly what the results of the three languages can and cannot tell us.

2. System architecture

The pipeline has three layers, shown in Figure 1. The **datalake** is the raw, immutable record of what was downloaded. The **datamarts** are derived, query-oriented stores that can always be rebuilt from the datalake. The **control layer** decides what happens next and records progress in plain files.

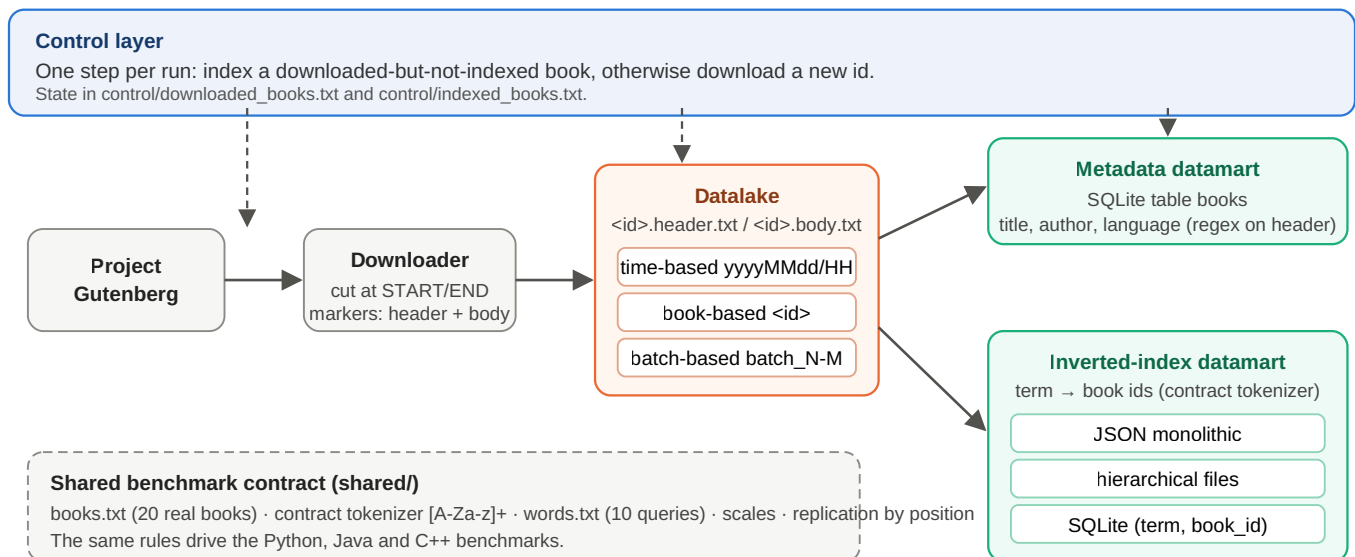


Figure 1. Pipeline architecture, common to the three implementations.

- **Datalake.** The downloader fetches `https://www.gutenberg.org/cache/epub/<id>/pg<id>.txt`, cuts the text at the `START` and `END OF THE PROJECT GUTENBERG EBOOK` markers and stores the part before the start marker as `<id>.header.txt` and the text between the markers as `<id>.body.txt`. Three directory layouts are implemented and benchmarked (Section 3.2); the live pipeline writes the time-based one, `datalake/yyyyMMdd/HH/`.

- **Metadata datamart.** Title, author and language are extracted from each header with regular expressions and stored in a SQLite table `books` keyed by `book_id` (Python and C++ also keep the paths of the two files).
- **Inverted-index datamart.** Each body is tokenized (lowercase, then maximal runs of `[A-Za-z]`; no stemming, no stop-word removal) and every unique term is mapped to the list of books that contain it. Three physical representations of the same index are implemented: one monolithic JSON file, a hierarchical layout of small files, and a SQLite table `inverted_index(term, book_id)` with a composite primary key.
- **Control layer.** Each run performs one step: if some downloaded book has not been indexed yet it is extracted and indexed, otherwise a not-yet-downloaded book id is chosen and downloaded. Progress is stored in `control/downloaded_books.txt` and `control/indexed_books.txt`, so an interrupted run resumes without repeating work.

	Python	Java	C++
Author	Aimar Daniel Alejandro Santana	Amado Artiles Rodríguez	Pablo Martínez Suárez
Stack	Python 3.9+; standard library (<code>sqlite3</code> , <code>json</code>) and <code>requests</code>	Java 17 target (benchmarked on JDK 24.0.1); Maven; <code>sqlite-jdbc</code> , Jackson, JMH 1.37	C++17; CMake/Make; <code>libcurl</code> , <code>SQLite3</code> , <code>nlohmann/json</code>
Tests	77 pytest tests, no network or external database needed	JUnit tests of the tokenizer; each benchmark checks its own results (row counts, complete recovery)	16 test suites, 119 assertions (CTest)
Index in the live pipeline	updates all three structures on every indexing step	writes the SQLite index	updates all three structures on every indexing step
Benchmarks	plain Python timing; <code>tracemalloc</code> for memory	JMH with GC profiler for the index; wall-clock for datalake and metadata	wall-clock timing; process resident memory in isolated worker processes

3. Design decisions

3.1 A shared benchmark contract

Three independent implementations are only comparable if they are fed the same data and asked the same questions. We therefore agreed a contract (`shared/CONTRACT.md`) that every language follows:

- **Dataset.** The same 20 real Gutenberg books (`shared/books.txt`). Larger volumes are *synthetic replications* of these 20 books under new ids, so the content is real text of real size.
- **Replication rule (Section 4.1 of the contract).** The synthetic book at 0-indexed *position* `i` uses the real book `books.txt[i mod 20]`. The rule is keyed on position, not on the numeric id, so it cannot drift if a start id changes. A consequence is that the vocabulary never grows (about 35,000 distinct terms at every scale) and only each term's postings list grows, so the experiments stress postings volume. The residual risk is a missing book silently shortening the cycle; the Java benchmarks fail fast in that case, and the vocabulary sizes of Python (35,008) and C++ (35,007) show that both used all 20 books.
- **Tokenizer.** Lowercase first, then maximal runs of `[A-Za-z]`.
- **Query workload.** Ten fixed words (`shared/words.txt`): `time love heart nothing death truth fortune monster wonderland darcy`, covering common, medium and rare terms.
- **Scales.** 100, 1,000 and 10,000 books. The first version of the contract used 1,000 / 10,000 / 100,000, but the largest scale proved impractical (about 11 hours of runtime on one machine), so the three scales were shifted down. The metadata benchmark adds a fourth scale, 100,000 books, because it touches a single SQLite file.
- **What to report.** Datalake: write throughput, lookup cost, incremental-processing cost, recovery behaviour and storage overhead. Inverted index: indexing speed, query performance, update performance, memory and disk usage, scalability. Metadata: insertion speed and the two query types (Section 4.4).

3.2 Datalake layout

Three layouts were implemented in all languages and benchmarked: **time-based** (yyyyMMdd/HH/, as recommended by the specification), **book-based** (one directory per book) and **batch-based** (directories of fixed id ranges, `batch_<start>-<end>`). **Decision: the live pipeline of the three implementations writes the time-based layout.** Section 4.2 shows that the layouts are equivalent in write speed, so the choice rests on structure: the time-based layout needs the fewest directories, its directory is the natural unit of incremental processing (what arrived in a given hour), and, because a directory only holds the books ingested in one hour, its size is bounded by the ingestion rate and not by the size of the corpus. The benchmark deliberately ingests everything within one hour, which is the worst case for this layout (10,000 files in one directory). If ingestion became bursty, the batch-based layout, which bounds every directory at 1,000 files at essentially the same cost, is the natural alternative.

3.3 Metadata datamart

Metadata is small, tabular and queried by key or by a text field, so a relational store is the natural fit. We use SQLite: a single file, no server process, transactions, and a primary-key B-tree on `book_id` that makes lookups by id independent of table size. Author search uses `LIKE '%author%'`, which cannot use an index and is a full scan; Section 4.4 quantifies that cost.

3.4 Inverted-index structures

Three representations of the same term → books mapping were compared. The **monolithic JSON** keeps everything in one file: trivial to write and, once loaded, a hash-map lookup, but it must be fully loaded before use and fully rewritten on every update. The **hierarchical** layout splits the index into many small files, so a lookup or an update touches only the files involved. **SQLite** stores one row per (term, book) posting under a composite primary key, which gives indexed lookups and cheap transactional updates at the price of a larger file. **Decision:** SQLite is the reference structure for serving queries and for incremental updates, because it combines the cheapest updates with indexed lookups and needs no load step; the monolithic JSON is kept as the small-corpus baseline and the hierarchical files as an engine-free alternative. Section 4.3 gives the evidence and its limits.

3.5 From MongoDB to SQLite

The third index structure was originally MongoDB in Python and C++. Midway through the stage the three of us replaced it with SQLite in every language: MongoDB proved heavy and unreliable on one member's machine, Java already used SQLite, and a cross-language comparison is only meaningful if the third structure is the same everywhere. All results in this report use SQLite.

4. Benchmarks and results

4.1 Experimental setup and comparability

Every number below comes from the JSON files committed in the repository (`python/datamarts/`, `c++/datamarts/`, `java/datamarts/` and `java/results.json`). The three implementations were run by their authors on **different computers**: Python on Windows 11, Java on Windows (JDK 24.0.1 HotSpot for the JMH index benchmark) and C++ on macOS. CPU, RAM and disk type were not recorded. Each data point is a single run, without repetitions or confidence intervals.

What “write throughput” means here. The assignment asks for download and write throughput. The benchmarks measure the *write* half: splitting and storing books that are already available locally (the 20 contract books, replicated to the requested scale). Downloading 10,000 books would mostly measure the network and the rate limits of Project Gutenberg, not the storage layout under study, and could not be repeated identically by the three authors. The downloader itself is implemented and tested in the three languages and is what the control layer uses in the live pipeline.

Writing the contract did not make every metric identical in practice. Reviewing the benchmark code of each language showed where the three measure the same thing and where they do not; Table 1 lists the differences, and the rest of the section uses them to decide which comparisons are valid.

Metric	Python	Java	C++	Comparable across languages?
Datalake write, incremental detection	All: write N books of 2 files each; detect which of the second half of the ids plus N/10 new ones are new, then write those; content by position			Yes (machines differ)
Datalake lookup	resolve the path and read the body	resolve the path and read the body	resolve the path and check that the files exist (no read)	Python and Java only
Datalake recovery	crash after 50%, resume skipping existing files	same as Python	delete the last 10%, recover those	Correctness only; cost not comparable
Batch size; depth	1,000 ids per batch; depth relative to the layout root	same as Python	500 ids per batch; counts the components of the absolute path	Directory counts of batch-based and “max depth” not comparable with C++
Index build	scan the datalake files, tokenize, build the index and write it	serialize an already built in-memory map	serialize an already built in-memory map	Java and C++ only; Python also includes tokenization
JSON lookup	in memory after a one-off cold load (reported separately)	read and parse the file on every lookup	in memory after a one-off cold load (reported separately)	Python and C++ only
SQLite lookup	All: indexed query for each of the 10 words, averaged per word. Python and C++: one pass	Java: JMH steady-state mean over many repetitions		Approximately
Hierarchical granularity	one file per term (about 35,000)	one file per first letter	one file per term (about 35,000)	Python and C++ only
Index update	add one full book (all its terms)	not measured	add a document of 3 words	No (trend within a language only)
Memory	peak traced allocation during the build	bytes allocated per operation (JMH)	resident set size of an isolated process	Three different metrics
SQLite index settings	defaults	<code>synchronous=OFF</code> , in-memory journal	<code>synchronous=OFF</code> , in-memory journal	Affects build and update
Metadata insertion	one connection and commit per row	one connection and commit per row	one transaction for all rows	Python and Java only
Metadata data and scales	20 real headers replicated by position (Dickens 15% of rows, Carroll 5%); 4 scales	same as Python	10 invented title/author seeds (Dickens 30%, Carroll 10%); 3 scales (no 100,000)	Shape of the curves, not magnitudes

Table 1. What each language measures. Green: same procedure; amber: partly; red: not comparable.

Consequence for the reading of the results. Absolute times are never ranked across languages unless the procedure is the same *and* the machine effect is discussed. The valid cross-language statements are (a) the same *qualitative* behaviour in all languages (which is what supports a design decision), (b) comparisons between Python and Java, which were both run on Windows, and (c) the metrics marked green or amber above. Values affected by a deviation are marked with * (C++ procedure differs), † (Java reads the file on every lookup) or ‡ (Python build includes tokenization) in the tables.

4.2 Datalake structure

Table 2 gives the results at 10,000 books; Table 3 the write throughput at the three scales; Figure 2 compares the layouts. Complete tables for every scale are in Appendix B.

Storage overhead. Every book is stored as two files (header and body), so any layout holds 2N files; what changes is how they are distributed over directories (columns *Dirs* and *Max files/dir* of Table 2). No layout needs

auxiliary structures beyond the two control files of the control layer (`downloaded_books.txt` and `indexed_books.txt`), which contain one book id per line (about 6 bytes per book, i.e. roughly 60 KB for 10,000 books) and are the same for the three layouts, so they do not affect the comparison.

Layout	Lang.	Write (s)	Books/s	Lookup (ms)	Dirs	Max files/dir	Incr. detect (s)	Incr. write (s)	Recovery (s)
time_based	Python	49.55	201.8	15.2	2	10,000	0.276	4.20	22.14
	Java	51.91	192.7	2.84	2	10,000	0.501	5.02	25.50
	C++	11.67	856.8	0.0176*	2	10,000	0.012	1.18	4.00*
book_based	Python	49.64	201.4	13.4	10,000	1	0.280	4.86	24.78
	Java	49.47	202.1	2.45	10,000	1	0.465	5.12	25.09
	C++	12.37	808.4	0.0129*	10,000	1	0.040	1.15	8.24*
batch_based	Python	42.81	233.6	12.5	10	1,000	0.175	3.74	22.42
	Java	49.52	201.9	2.41	10	1,000	0.436	5.10	25.54
	C++	11.47	872.0	0.0147*	20	500	0.032	1.05	4.37*

Table 2. Datalake results at 10,000 books. * C++: lookup only checks existence, recovery restores 10% of the books and C++ uses batches of 500 ids (20 directories instead of 10); see Table 1.

Layout	Python			Java			C++		
	100	1,000	10,000	100	1,000	10,000	100	1,000	10,000
time_based	207.0	225.3	201.8	184.7	184.3	192.7	1,015.5	885.0	856.8
book_based	225.3	199.4	201.4	201.9	189.2	202.1	858.2	785.2	808.4
batch_based	190.7	208.4	233.6	185.1	198.4	201.9	881.6	890.4	872.0

Table 3. Write throughput (books per second) at 100 / 1,000 / 10,000 books.

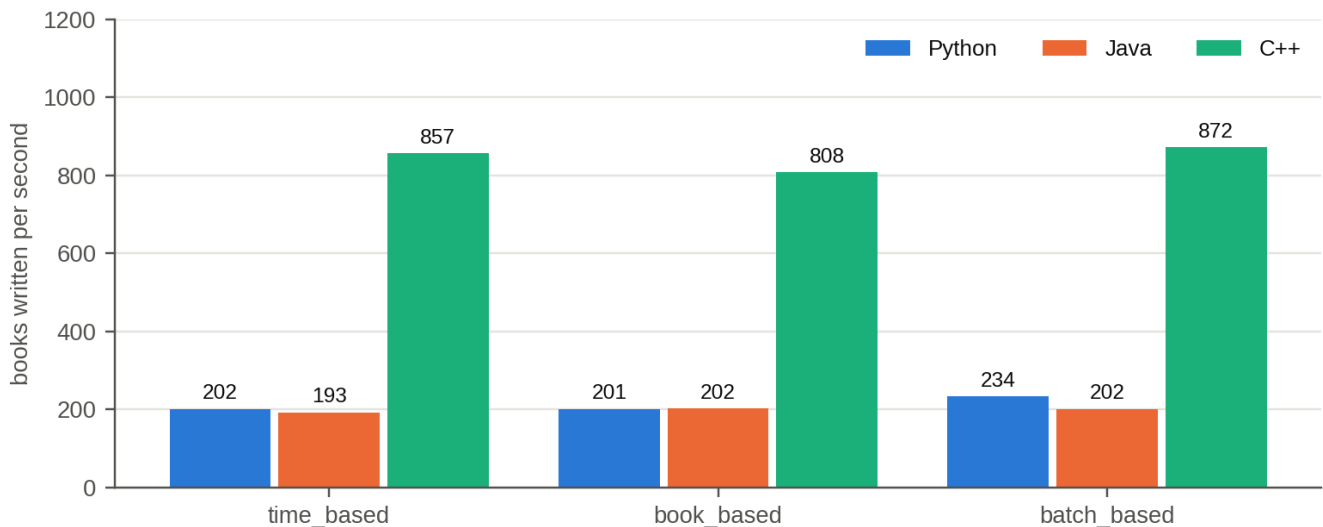


Figure 2. Write throughput at 10,000 books. Different machines: compare layouts within a language, not languages with each other.

- **The layout does not decide the write speed.** In each language the slowest of the three layouts is at most 14% below the fastest at 10,000 books, and no layout wins consistently: batch-based is fastest in Python (234 books/s against 202 and 201), time-based is slowest in Java (193 against 202 and 202) and book-based is slowest in C++ (808 against 857 and 872). Throughput is flat across scales (Table 3), so the total time is linear in the number of books: the cost is dominated by creating two files per book.
- **The layouts do differ in structure.** At 10,000 books time-based uses 2 directories with 10,000 files in one of them, book-based uses 10,000 directories with one file pair each, and batch-based bounds every directory at its

batch size. This is the real trade-off between the layouts, and it is identical in Python and Java (the structural columns of the two are equal in all 9 layout and scale combinations).

- **Python and Java agree; C++ is about four times faster.** On the same operating system the two interpreted/managed implementations write at the same speed (193–234 books/s), which is expected when the time is spent in the file system. C++ writes 3.7–4.4× faster, but it ran on a different operating system and file system, so the data cannot separate the effect of the language from that of the platform; since the workload is I/O-bound, we expect the platform to explain most of it, but we did not test that.
- **Incremental ingestion is cheap in every language.** Detecting which of the 6,000 candidates are new costs 1%, 7% and 10% of the time needed to write the 1,000 new books (C++, Python and Java, time-based layout at 10,000 books), so the check that avoids duplicates costs at most a tenth of the work it saves.
- **Recovery is correct in all three languages.** After a simulated interruption the resumed run restored exactly the missing books and left exactly N body files, with no duplicates or losses, in all 27 combinations of language, layout and scale. The recovery cost is not compared because the C++ procedure is different (Table 1).
- **Lookup.** Python and Java, which both read the body, take 13–15 ms and 2.0–2.8 ms per lookup, with no meaningful difference between layouts. The C++ values (about 0.01 ms) only test that the file exists and must not be compared with them.

Conclusion for the datalake. Write speed, lookup cost and incremental detection do not separate the layouts; the choice is a structural one. We keep the time-based layout (bounded by the ingestion rate, fewest directories) and recommend batch-based if ingestion becomes bursty.

4.3 Inverted-index structure

Table 4 summarizes the three structures in each language at 10,000 books; Figure 3 shows how the SQLite lookup scales; Appendix B gives all scales.

Structure	Lang.	Build (s)	Lookup / word (ms)	Update	Memory	Disk (MB)
json_monolithic	Python	2,021.9‡	0.0048	48.0 s	4,627 MB peak	534
	Java	2.50	3,309†	n/m	21,607 MB alloc./10 words	n/m
	C++	5.08	0.0064	11.9 s	1,307 MB peak	801
hierarchical	Python	1,487.3‡	11.9	28.4 s	4,109 MB peak	534
	Java	2.04	106†	n/m	989 MB alloc./10 words	n/m
	C++	6.44	0.458	4 ms	344 MB peak	467
sqlite	Python	1,582.2‡	34.5	1.8 s	8,385 MB peak	2,955
	Java	111.7	1.08	n/m	2.46 MB alloc./10 words	n/m
	C++	80.2	4.44	2 ms	347 MB peak	4,225

Table 4. Inverted index at 10,000 books. ‡ Python's build scans the datalake and tokenizes the books; Java's and C++'s time only the serialization of an index already built in memory. † Java parses the file on every lookup (3.3 s per word for the monolithic file); Python and C++ look up in memory after a cold load of 10.6 s and 7.0 s. Update: Python adds one full book, C++ a 3-word document, Java did not measure it (n/m). Memory: three different metrics (Table 1). Disk: Java did not record it.

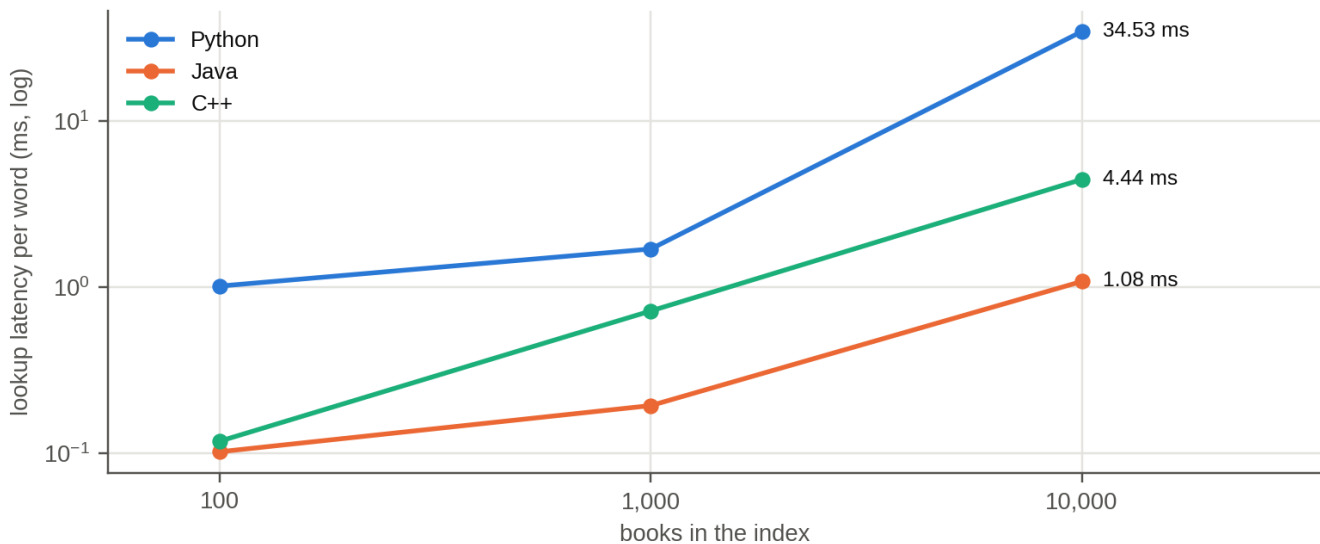


Figure 3. SQLite index: lookup latency per word against the number of books (log-log). Java is a JMH steady-state mean; Python and C++ are a single pass; machines differ.

- **Updating: SQLite wins, and the monolithic file collapses.** In both languages that measured it, the monolithic JSON is by far the most expensive to update because the whole file is rewritten: at 10,000 books Python needs 48 s and C++ 11.9 s, against 1.78 s and 2.0 ms for SQLite. The ranking SQLite < hierarchical < monolithic is the same in both. The absolute values are not comparable (Python adds a whole book, C++ a 3-word document), but the ranking and the growth of the monolithic cost with the index size are.
- **Reading: in-memory JSON is fastest but has to be loaded.** Once loaded, the monolithic JSON answers in microseconds in Python (4.8 μ s) and C++ (6.4 μ s), but only after a cold load of 10.6 s and 7.0 s and with a resident footprint that grows with the index (4.5 GB in Python, 1.3 GB in C++ at 10,000 books). Java, which re-reads the file on every lookup, shows the price of having no load step: 3.3 s per word. SQLite needs no load step and costs 34.5, 4.4 and 1.1 ms per word in Python, C++ and Java.
- **SQLite lookups grow with the scale in all three languages** ($\times 34$, $\times 38$ and $\times 11$ from 100 to 10,000 books in Python, C++ and Java). Because the vocabulary is fixed by the replication rule, a term appears in a fixed fraction of the books, so its postings list, and the number of rows a lookup must return, grows with the scale; this is result-set cost, not a deeper B-tree.
- **Disk: SQLite is about five times larger.** The SQLite file is 5.5 $\times$ the monolithic JSON in Python and 5.3 $\times$ in C++ at 10,000 books (2.9 GB and 4.1 GB), because it stores one row with its B-tree overhead per (term, book) posting instead of a compact list of ids. The factor is consistent across the two languages that measured it.
- **Build: once tokenization is excluded, SQLite is the slowest to write.** The two languages that time only the serialization of an in-memory index agree: SQLite takes 80 s in C++ and 112 s in Java at 10,000 books, 16 $\times$ and 45 $\times$ the monolithic JSON (5.1 s and 2.5 s), because every posting becomes a row in a B-tree. Python's figures (1,582 s for SQLite against 2,022 s for the monolithic JSON) are dominated by reading and tokenizing 10,000 files, which is why they cannot be compared with the other two and why the structures are almost indistinguishable in them. The hierarchical layout is cheap to build in Java (2.0 s) because it writes one file per letter, whereas C++ writes one per term (6.4 s).
- **Memory.** The three metrics (Table 1) cannot be compared across languages, but they show the same trend within each: footprint grows roughly linearly with the number of books for the structures that hold the index in memory (Python: 68 $\rightarrow$ 4,627 MB; C++: 48 $\rightarrow$ 1,307 MB for the monolithic JSON). Java's allocation per 10-word batch makes the cost of its read-and-parse strategy explicit: 21.6 GB for the monolithic file and 2.5 MB for SQLite.
- **The largest cross-language gap should not be attributed to the language.** At 10,000 books the SQLite lookup is 8 $\times$ slower in Python than in C++ and 32 $\times$ slower than in Java. All three call the same SQLite engine

through a different binding, and the procedures differ (JMH steady state against a single pass) and the machines differ, so we treat this as an open question to be settled by running the three on one machine with one protocol.

Conclusion for the inverted index. No structure wins everywhere, but the qualitative pattern is the same in every language: the monolithic JSON is trivial to build and fast once in memory but unusable for incremental updates and costly in memory; SQLite is the most expensive to write and the largest on disk (about five times) but needs no load step and has the cheapest updates; the hierarchical layout sits in between. For a search engine that keeps ingesting books, updates and the absence of a load step matter more than build time, which supports SQLite as the reference structure.

4.4 Metadata datamart

Lang.	Books	Insert (s)	Rows/s	find_by_id (ms)	Dickens (ms)	Carroll (ms)	DB (KB)
Python	100	0.694	144	0.53	0.51	0.51	24
	1,000	4.97	201	0.40	0.83	0.82	132
	10,000	51.1	196	0.65	8.00	5.28	1,224
	100,000	526	190	0.57	57.38	34.64	12,164
Java	100	0.786	127	1.80	2.14	1.75	16
	1,000	7.09	141	1.57	2.54	2.35	72
	10,000	69.6	144	1.54	8.38	6.55	620
	100,000	675	148	1.65	65.12	43.84	6,112
C++	100	0.0089	11,205*	0.23	0.23*	0.19*	28
	1,000	0.0038	260,827*	0.15	0.28*	0.24*	168
	10,000	0.0186	536,940*	0.14	1.45*	1.10*	1,564

Table 5. Metadata datamart. * C++ inserts all rows in one transaction and its data contain Dickens in 30% and Carroll in 10% of the rows (15% and 5% in Python and Java). Python and Java store different columns (six and four), so database sizes differ.

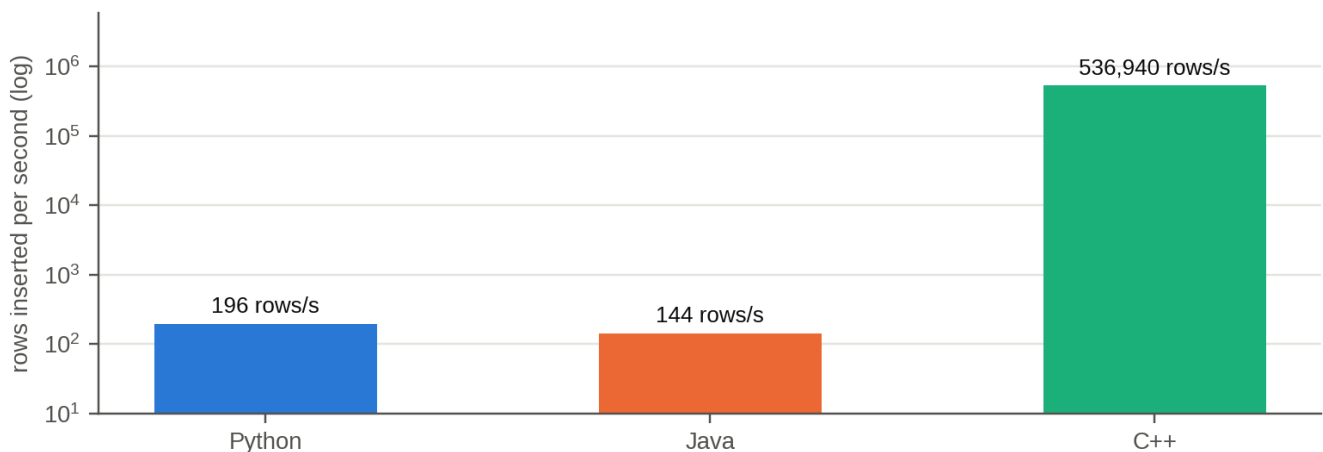


Figure 4. Insertion throughput at 10,000 rows. Python and Java commit one transaction per row; C++ commits once for all rows.

- **Transaction batching matters far more than the language.** Python and Java, which open a connection and commit for each row exactly as the pipeline does, insert at a flat 144–201 rows/s (Python) and 127–148 rows/s (Java) from 100 to 100,000 rows: the per-call cost of the connection and the commit dominates and does not depend on the table size, so the total time is linear (8.8 and 11.3 minutes for 100,000 rows). C++, which inserts in a single transaction, reaches 536,940 rows/s, 2,745× Python and 3,737× Java. A gap of this size comes from

the commit strategy, not from the language, and it identifies a clear improvement for the Python and Java pipelines: bulk loads should use a single transaction.

- **Primary-key lookups are independent of the table size.** `find_by_id` stays at 0.4–0.6 ms (Python), 1.5–1.8 ms (Java) and 0.14–0.23 ms (C++) over three orders of magnitude of table size, as expected from a B-tree. The absolute level reflects the per-call connection cost of each binding (Java is the highest).
- **Author search is a full scan and grows with the table.** Between 10,000 and 100,000 rows the author queries grow $\times 7.2$ in Python and $\times 7.8$ in Java for Charles Dickens, to 57 and 65 ms, and the repeated author is slower than the unique one (35 and 44 ms) because it returns three times as many rows. C++ shows the same linear growth at the scales it measured (1.45 ms at 10,000 rows) but on a table with twice the proportion of matching rows, so its magnitudes are not comparable.
- **Python against Java.** On the same workload, same method and same operating system, Java takes 1.13–1.43 $\times$ Python's insertion time and 1.14 $\times$ and 1.27 $\times$ its author-query time at 100,000 rows, so the two curves have the same shape with Java somewhat slower, plausibly because of the cost of opening a JDBC connection per call (not measured separately; the two runs used different computers).

Conclusion for the metadata. SQLite is adequate: constant-time lookups by id and, for a catalogue of Gutenberg size (tens of thousands of books), an author search of at most a few tens of milliseconds. The two weaknesses are known and fixable: per-row commits make bulk loads slow, and the author search needs an index (or full-text search) if the catalogue grows.

4.5 What the three languages tell us

Question	What the data supports
Do the language implementations behave the same way?	Yes, qualitatively. The same structural results (directories, files per directory, incremental and resumed counts) appear in Python and Java in all 9 datalake combinations, every language recovers correctly in all 27 combinations, and all three languages order the index structures the same way for update cost, disk size and load behaviour.
Is one language faster?	Not demonstrated. Python and Java, both on Windows, are equal on the I/O-bound datalake and similar on the SQLite metadata (Java 1.1–1.4 $\times$ slower to insert). C++ is about four times faster on the datalake, but on a different operating system; the I/O-bound workload suggests the platform is the main reason, which was not tested. The largest differences in the study (batched metadata insertion, Python's SQLite lookup) come from the procedure or are unexplained, not from the language.
What would make a ranking valid?	Running the three implementations on one machine with one protocol: the same update workload, the same recovery procedure, the same metadata data and commit strategy, a common memory metric, and several repetitions with confidence intervals.

5. Conclusions and future improvements

Conclusions. We built the Stage 1 data layer three times, with the same architecture, and benchmarked it with a shared contract. For the datalake, the three layouts are equivalent in write speed, lookup and incremental cost in every language, so the choice is structural; we keep the time-based layout, whose directory size is bounded by the ingestion rate, and recommend batch-based for bursty ingestion. For the inverted index, SQLite is the best reference structure for a system that keeps ingesting books: it has the cheapest updates and needs no load step, at the price of the slowest build and about five times the disk of the JSON structures; the monolithic JSON is only acceptable for small corpora and the hierarchical layout is an engine-free middle ground. For the metadata, SQLite gives constant-time lookups by id, a linear but small author-search cost at the scales of a real catalogue, and shows that per-row commits, not the language, limit insertion speed.

The study also taught us a methodological lesson: writing a shared contract is necessary but not sufficient. Several metrics ended up measured differently in each language (Table 1), which prevents ranking the languages on them; recognizing and reporting this is part of the result.

Future improvements.

- **Make the benchmarks strictly comparable:** one machine, one protocol and repetitions with confidence intervals; align the C++ metadata benchmark (real headers, contract author proportions, the 100,000 scale), recovery procedure and batch size; use one update workload and one memory metric; add the index update and disk size to the Java benchmark.
- **Pipeline:** load metadata in batched transactions; give the Java JSON storages an in-memory cache; add an index (or SQLite full-text search) for author queries; consider term-level pagination of very long postings lists.
- **Next stages:** parallel downloaders and indexers, term frequencies and ranking (TF-IDF or BM25), a query engine with Boolean operators, and a corpus beyond 10,000 books, where the directory-size and disk-footprint trade-offs found here will start to matter.

Appendix A. Repository and reproduction

Repository: https://github.com/TheScratchers/stage_1. Each implementation has its own folder and README with setup instructions; `shared/` holds the contract (`CONTRACT.md`, `books.txt`, `words.txt`) and `data/sample_books/` a small sample dataset to try the pipeline. Raw results: `python/datamarts/`, `c++/datamarts/`, `java/datamarts/` and `java/results.json`.

```
# Python (from python/)
pip install -r requirements.txt
python src/download_shared_dataset.py                # the 20 contract books
python src/benchmark_datalake.py 100,1000,10000
python src/benchmark_inverted_index.py 100,1000,10000
python src/benchmark_metadata.py 100,1000,10000,100000
python -m pytest tests/

# Java (from java/)
mvn clean package
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.DownloadContractBooks
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.BenchmarkRunner      # index (JMH)
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.DatalakeBenchmark
java -cp target/benchmarks.jar com.thescratchers.searchengine.benchmark.MetadataBenchmark

# C++ (from c++/)
cmake -B build -S . && cmake --build build
ctest --test-dir build --output-on-failure
```

The C++ benchmark options are described in `c++/README.md`.

Appendix B. Complete results

B.1 Datalake (all layouts and scales)

Python

Layout	Books	Write (s)	Books/s	Lookup (ms)	Dirs	Max files/dir	New found	Detect (s)	Write new (s)	Resumed	Recovery (s)
time_based	100	0.48	207.0	14.4	2	100	10	0.0028	0.034	50	0.233
	1,000	4.44	225.3	12.9	2	1,000	100	0.0266	0.454	500	2.426
	10,000	49.55	201.8	15.2	2	10,000	1,000	0.2756	4.199	5,000	22.141
book_based	100	0.44	225.3	13.4	100	1	10	0.0020	0.038	50	0.260
	1,000	5.01	199.4	13.6	1,000	1	100	0.0282	0.455	500	2.427

Layout	Books	Write (s)	Books/s	Lookup (ms)	Dirs	Max files/dir	New found	Detect (s)	Write new (s)	Resumed	Recovery (s)
batch_based	10,000	49.64	201.4	13.4	10,000	1	1,000	0.2797	4.861	5,000	24.782
	100	0.52	190.7	13.1	1	100	10	0.0019	0.034	50	0.198
	1,000	4.80	208.4	13.4	1	1,000	100	0.0298	0.568	500	2.528
	10,000	42.81	233.6	12.5	10	1,000	1,000	0.1748	3.740	5,000	22.421

Java

Layout	Books	Write (s)	Books/s	Lookup (ms)	Dirs	Max files/dir	New found	Detect (s)	Write new (s)	Resumed	Recovery (s)
time_based	100	0.54	184.7	2.18	2	100	10	0.0045	0.070	50	0.243
	1,000	5.42	184.3	2.63	2	1,000	100	0.0937	0.676	500	2.720
	10,000	51.91	192.7	2.84	2	10,000	1,000	0.5007	5.021	5,000	25.502
book_based	100	0.50	201.9	2.04	100	1	10	0.0050	0.060	50	0.270
	1,000	5.28	189.2	2.01	1,000	1	100	0.0437	0.591	500	2.589
	10,000	49.47	202.1	2.45	10,000	1	1,000	0.4652	5.120	5,000	25.088
batch_based	100	0.54	185.1	2.18	1	100	10	0.0047	0.062	50	0.254
	1,000	5.04	198.4	2.23	1	1,000	100	0.0466	0.524	500	2.610
	10,000	49.52	201.9	2.41	10	1,000	1,000	0.4361	5.101	5,000	25.538

C++

Layout	Books	Write (s)	Books/s	Lookup (ms)	Dirs	Max files/dir	New found	Detect (s)	Write new (s)	Resumed	Recovery (s)
time_based	100	0.10	1,015.5	0.0124	2	100	10	0.0002	0.009	10	0.019
	1,000	1.13	885.0	0.0146	2	1,000	100	0.0014	0.131	100	0.171
	10,000	11.67	856.8	0.0176	2	10,000	1,000	0.0123	1.184	1,000	3.996
book_based	100	0.12	858.2	0.0094	100	1	10	0.0003	0.012	10	0.014
	1,000	1.27	785.2	0.0121	1,000	1	100	0.0039	0.114	100	0.125
	10,000	12.37	808.4	0.0129	10,000	1	1,000	0.0400	1.153	1,000	8.241
batch_based	100	0.11	881.6	0.0106	1	100	10	0.0003	0.010	10	0.011
	1,000	1.12	890.4	0.0127	2	500	100	0.0033	0.112	100	0.123
	10,000	11.47	872.0	0.0147	20	500	1,000	0.0324	1.050	1,000	4.366

Lookup: Python and Java read the body, C++ checks existence. Recovery: Python and Java resume after a crash at 50%, C++ restores 10% of the books. "New found": new books detected among the incremental candidates.

B.2 Inverted index (all structures and scales)

Python

Structure	Books	Build (s)*	Cold load (s)	Lookup (ms)	Update (s)	Peak memory (MB)	Disk (MB)
json_monolithic	100	19.9	0.14	0.0029	0.490	68.0	5.8
	1,000	184	0.89	0.0017	4.006	413.5	53.8
	10,000	2,022	10.56	0.0048	47.975	4,626.8	534.1
hierarchical	100	43.5	—	1.88	6.857	68.0	5.3
	1,000	173	—	8.34	15.890	375.9	53.3
	10,000	1,487	—	11.9	28.443	4,109.0	533.5
sqlite	100	17.4	—	1.01	0.222	96.8	28.0
	1,000	167	—	1.70	0.332	790.5	297.0

Structure	Books	Build (s)*	Cold load (s)	Lookup (ms)	Update (s)	Peak memory (MB)	Disk (MB)
	10,000	1,582	–	34.5	1.779	8,385.0	2,955.5

C++

Structure	Books	Build (s)	Cold load (s)	Lookup (ms)	Update (s)	Process memory (MB)	Disk (MB)
json_monolithic	100	0.0514	0.09	0.0010	0.138	47.5	8.7
	1,000	0.389	0.70	0.0017	1.039	333.3	80.7
	10,000	5.08	7.04	0.0064	11.915	1,307.3	801.0
hierarchical	100	2.72	–	0.130	0.00037	8.2	4.7
	1,000	2.94	–	0.100	0.00069	39.9	46.7
	10,000	6.44	–	0.458	0.00394	344.4	466.9
sqlite	100	0.742	–	0.118	0.00068	10.1	40.1
	1,000	7.20	–	0.718	0.00069	42.8	406.8
	10,000	80.2	–	4.44	0.00198	347.4	4,225.4

Java

Structure	Books	Build (s)	Lookup per word (ms)	Allocated per 10-word batch (MB)
json_monolithic	100	0.0224	59.8	227.28
	1,000	0.181	288	2,194.29
	10,000	2.50	3,309	21,606.59
hierarchical	100	0.0666	2.73	12.08
	1,000	0.316	10.8	102.08
	10,000	2.04	106	988.53
sqlite	100	0.914	0.102	0.02
	1,000	7.90	0.193	0.23
	10,000	112	1.08	2.46

* Python's build includes scanning and tokenizing the datalake. Python and C++ JSON lookups are in memory after the cold load; Java's JSON lookups parse the file each time. Python update = add one book; C++ update = add a 3-word document. Memory metrics differ by language (Table 1).